

Exp No.: 6 Implementing Fabric Channels for Privacy

Aim

To create and configure multiple channels in a Hyperledger Fabric network, deploy chaincode on a specific channel, and demonstrate how Fabric channels provide data privacy and isolation between organizations.

Software Requirements

- ☐ Kali Linux / Ubuntu
- ☐ Docker
- ☐ Docker Compose
- ☐ Node.js (Version 18 or later)
- ☐ npm
- ☐ Git
- ☐ Hyperledger Fabric Samples
- ☐ Hyperledger Fabric Binaries
- ☐ Hyperledger Fabric Docker Images
- ☐ Visual Studio Code (Optional)

Hardware Requirements

- ☐ Processor: Intel Core i3 or above
- ☐ RAM: Minimum 8 GB
- ☐ Storage: Minimum 20 GB Free Disk Space
- ☐ Internet Connection

Theory

A Fabric Channel is a private communication mechanism that allows a subset of organizations to maintain a separate ledger and execute transactions confidentially. Each

channel has its own blockchain ledger, smart contracts (chaincode), and world state database.

Organizations that are not members of a channel cannot access its ledger or transaction data. Channels enable enterprise applications to maintain confidentiality while sharing the same Hyperledger Fabric network.

Procedure

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```

Expected Output

Creating channel 'mychannel'

Starting peer0.org1

Starting peer0.org2

Starting orderer.example.com

Network Started Successfully

Step 3: Verify the Existing Channel
List the channels joined by the peer.
peer channel list

Expected Output
Channels peers has joined:

mychannel

Step 4: Create a New Private Channel
Create a second channel named privatechannel.
./network.sh createChannel -c privatechannel

Expected Output
Channel 'privatechannel' created successfully.

Step 5: Verify Available Channels
peer channel list

Expected Output
Channels peers has joined:

mychannel

privatechannel

Step 6: Deploy Chaincode on the Private Channel
Deploy the JavaScript Asset Transfer chaincode only to privatechannel.
./network.sh deployCC \
-c privatechannel \
-ccn basic \
-ccl javascript \
-ccp ../asset-transfer-basic/chaincode-javascript

Expected Output

Packaging Chaincode

Installing Chaincode

Approving Chaincode

Committing Chaincode

Chaincode committed successfully

Step 7: Initialize the Ledger

Initialize the ledger for the privatechannel.

peer chaincode invoke \

-o localhost:7050 \

--ordererTLSTLSHostnameOverride orderer.example.com \

--tls \

--cafile \$ORDERER_CA \

-C privatechannel \

-n basic \

-c '{"function":"InitLedger","Args":[]}'

Expected Output

Ledger Initialized Successfully

Step 8: Query Assets from the Private Channel

peer chaincode query \

-C privatechannel \

-n basic \

-c '{"Args":["GetAllAssets"]}'

Expected Output

```
[
{
  "ID":"asset1",
  "Owner":"Tomoko",
  "Color":"Blue"
},
{
  "ID":"asset2",
  "Owner":"Brad",
  "Color":"Red"
}
]
```

Step 9: Verify Channel Isolation

Attempt to query privatechannel data from mychannel.

```
peer chaincode query \
```

```
-C mychannel \
```

```
-n basic \
```

```
-c '{"Args":["ReadAsset","asset1"]}'
```

Expected Output

Asset data returned only from mychannel.

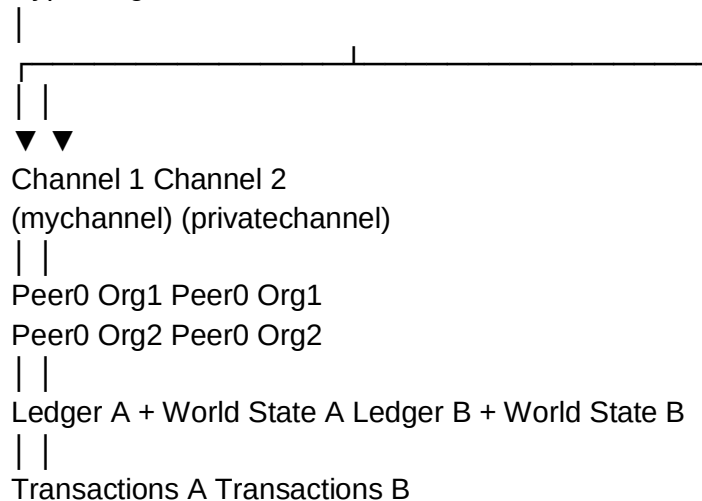
Data stored in privatechannel is not accessible through mychannel.

Step 10: Stop the Fabric Network

```
./network.sh down
```

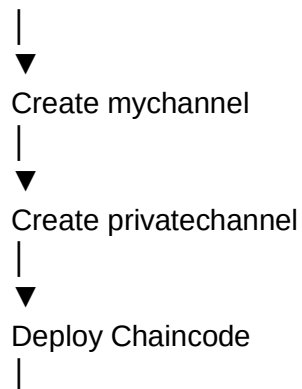
Architecture Diagram

Hyperledger Fabric Network



Flow Diagram

Start Fabric Network





Initialize Ledger



Query Assets



Verify Channel Isolation



Stop Network

Sample Output

Fabric Network Started Successfully

mychannel Created Successfully

privatechannel Created Successfully

Chaincode Installed Successfully

Ledger Initialized Successfully

Assets Retrieved Successfully

Privacy Verified Successfully

Network Stopped Successfully

```

[tharun@kali:]-~/fabric-dev/fabric-samples/asset-transfer-basic/application-gateway-javascript
$ npm start

> asset-transfer-basic@1.0.0 start
> node src/app.js

channelName: mychannel
chaincodeName: basic
mspid: Org1MSP
cryptoPath: /home/tharun/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com
keyDirectoryPath: /home/tharun/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/User@org1.example.com/msp/keystore
keyStorePath: /home/tharun/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/User@org1.example.com/msp/signcerts
tlsCertPath: /home/tharun/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
peerEndpoint: localhost:7051
peerHostAlias: peer0.org1.example.com

Step 4 - Expected behavior

--> Submit Transaction: InitLedger, function creates the initial set of assets on the ledger
--** Transaction committed successfully

--> Evaluate Transaction: GetAllAssets, function returns all the current assets on the ledger
--** Result: [
  {
    AppraisedValue: 300,
    Color: 'blue',
    ID: 'asset1',
    Owner: 'Tomoko',
    Size: 5,
    docType: 'asset'
  },
  {
    AppraisedValue: 400,
    Color: 'red',
    ID: 'asset2',
    Owner: 'Brad',
    Size: 5,
    docType: 'asset'
  },
  {
    AppraisedValue: 1500,
    Color: 'Green',
    ID: 'asset2a',
    Owner: 'Arun',
    Size: 8
  },
  {
    AppraisedValue: 500,
    Color: 'green',
    ID: 'asset3',
    Owner: 'Jin Soo',
    Size: 10,
    docType: 'asset'
  }
]

```

```

Querying chaincode definition on peer0.org1 on channel 'mychannel' ...
Attempting to query committed status on peer0.org1. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 2, Endorsement Plugin: escc, Validation Plugin: vscv, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel' ...
Attempting to query committed status on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 2, Endorsement Plugin: escc, Validation Plugin: vscv, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required

Step 7 — Install Node.js Dependencies
tharun@kali: ~/fabric-dev/fabric-samples/test-network
$ peer lifecycle chaincode querycommitted -c mychannel -n basic
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 2, Endorsement Plugin: escc, Validation Plugin: vscv, Approvals: [Org1MSP: true, Org2MSP: true]

tharun@kali: ~/fabric-dev/fabric-samples/test-network
$ node --version
v24.10.0
npm --version
11.16.0

tharun@kali: ~/fabric-dev/fabric-samples/test-network
$ cd ../asset-transfer-basic/application-gateway-javascript

tharun@kali: ~/fabric-dev/fabric-samples/asset-transfer-basic/application-gateway-javascript
$ pwd
/home/tharun/fabric-dev/fabric-samples/asset-transfer-basic/application-gateway-javascript

tharun@kali: ~/fabric-dev/fabric-samples/asset-transfer-basic/application-gateway-javascript
$ ls -ls
total 12
4 -rw-rw-r-- 1 tharun docker 208 Aug 20 17:00 eslint.config.mjs
4 -rw-rw-r-- 1 tharun docker 655 Aug 20 17:09 package.json
4 drwxrwxr-x 2 tharun docker 4096 Aug 20 17:09 src

tharun@kali: ~/fabric-dev/fabric-samples/asset-transfer-basic/application-gateway-javascript
$ ls -la
total 20
drwxrwxr-x 3 tharun docker 4096 Aug 20 17:09 .
drwxrwxr-x 13 tharun docker 4096 Aug 20 17:09 ..
-rw-rw-r-- 1 tharun docker 298 Aug 20 17:09 eslint.config.mjs
-rw-rw-r-- 1 tharun docker 157 Aug 20 17:09 gitignore
-rw-rw-r-- 1 tharun docker 19 Aug 20 17:09 .npmrc
-rw-rw-r-- 1 tharun docker 655 Aug 20 17:09 package.json
drwxrwxr-x 2 tharun docker 4096 Aug 20 17:09 src

Step 8 — Inspect app.js
Before running 'npm install', inspect the sample application in 'app.js'

tharun@kali: ~/fabric-dev/fabric-samples/test-network
$ ./network.sh deployCC \
  -cc lang javascript \
  -ccn basic \
  -cnp ../asset-transfer-basic/chaincode-javascript
Using docker and docker-compose to deploy chaincode
Deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1 ...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[]).package_id'
+ grep '"basic_1.0:1d4d445573112813889f87c307bc2b5fbee64c87b24c035bee15cbcc7f59b6295"'
+ test 0 -ne 0
basic_1.0:1d4d445573112813889f87c307bc2b5fbee64c87b24c035bee15cbcc7f59b629
scripts/envVar.sh: line 86: [: too many arguments
Chaincode is installed on peer0.org1
Install chaincode on peer0.org2 ...
Using organization 2
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[]).package_id'
+ grep '"basic_1.0:1d4d445573112813889f87c307bc2b5fbee64c87b24c035bee15cbcc7f59b6295"'
+ test 0 -ne 0
basic_1.0:1d4d445573112813889f87c307bc2b5fbee64c87b24c035bee15cbcc7f59b629
scripts/envVar.sh: line 86: [: too many arguments
Chaincode is installed on peer0.org2
++ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
++ sed -n '/version:/s/.*sequence: //; s/, Endorsement Plugin:.*//; p;'
+ COMMITTED_CC_SEQUENCE=1
+ res=0
++ peer lifecycle chaincode queryapproved --channelID mychannel --name basic
++ sed -n '/sequence:/s/.*sequence: //; s/, version:.*//; p;'

```

Result

The Hyperledger Fabric network was successfully configured with multiple channels. A private channel was created, chaincode was deployed, and ledger transactions were executed successfully. The experiment demonstrated that channel members can access only

the data belonging to their respective channels, ensuring privacy and confidentiality in a permissioned blockchain network